

Critical Evaluation and Reproduction of Machine Learning-Based Website Phishing Detection

Selected article: Towards Benchmark Datasets for Machine Learning Based Website Phishing Detection: An Experimental Study

by Abdelhakim Hannousse and Salima Yahiouche.

Data Science in Cybersecurity Project Report

Student: Moa'awiayh Hajajrah.
ID: 208038059

Table of Contents

Executive Summary

1. Summary of the Source
2. Critical Evaluation
3. Feature Engineering Analysis
4. Reproducibility Analysis
5. Experimental Results
6. Conclusions

References

Executive Summary

This report critically evaluates the article “Towards Benchmark Datasets for Machine Learning Based Website Phishing Detection: An Experimental Study” by Abdelhakim Hannousse and Salima Yahiouche. The article addresses a core cybersecurity problem: automatically identifying phishing websites before users submit credentials, financial information, or other sensitive data.

The selected article is suitable for this assignment because it presents a reproducible phishing-detection dataset, defines a machine-learning problem, extracts a rich feature set, evaluates several classifiers, and reports claims that can be reproduced and challenged experimentally. The project uses the released feature-vector dataset, referred to in the article as Dataset B, and reproduces the main model-level findings using Python, scikit-learn, and XGBoost.

The dataset used in the notebook contains 11,430 websites: 5,715 legitimate websites and 5,715 phishing websites. The raw dataset has 89 columns: one URL identifier, 87 extracted features, and one target column. After cleaning, removing identifier-like columns, duplicated columns, and one constant feature, the final modeling matrix contains 81 numeric features.

The reproduced experiments support the article’s main conclusion that Random Forest is a strong model for phishing detection. Random Forest achieved 96.24% accuracy, 96.25% F1-score, 96.59% recall, 99.34% ROC-AUC, and 92.48% MCC on the test set. These results are close to the article’s reported Random Forest accuracy of about 96.61% using hybrid features.

The project also extends the article by adding XGBoost which we added a newer model that hasn’t been tested. XGBoost achieved 96.68% accuracy and 96.68% F1-score, slightly outperforming Random Forest. This does not make the article wrong; rather, it shows that the article’s claim that Random Forest is best should be interpreted within the limited model set tested by the authors.

The feature-group results strongly support the article’s claim that external-service features are highly discriminative. With only seven external-service features, Random Forest achieved 93.66% accuracy, outperforming URL-only and content-only models. The URL + External combination achieved 96.41% accuracy, slightly exceeding the all-feature Random Forest result of 96.24%, which further supports the importance of external-service signals.

From a cybersecurity perspective, the most important finding is that balanced benchmark accuracy is not sufficient for deployment decisions. When the test set was made more realistic with approximately 5% phishing prevalence, Random Forest still maintained high recall of 96.67%, but precision dropped to 55.24%. This means that the model continued to detect most phishing websites, but many phishing alerts would be false positives in a real-world environment.

Overall, the article is mostly supported by the reproduced experiments. Its main strengths are dataset reproducibility, meaningful feature engineering, and strong Random Forest performance. Its main limitations are the limited model comparison, reliance on external-service features, balanced benchmark setting, and lack of deep deployment-focused analysis. This report therefore concludes that the paper is strong and reproducible, but not exhaustive.

1. Summary of the Source

1.1 Selected Source

The selected source is the article “Towards Benchmark Datasets for Machine Learning Based Website Phishing Detection: An Experimental Study” by Abdelhakim Hannousse and Salima Yahiouche. The article was released as a 2020 preprint and focuses on machine-learning-based website phishing detection.

This source was selected because it satisfies the assignment requirements: it addresses a cybersecurity problem, proposes a data-science solution, provides a dataset and implementation-related resources, and reports claims that can be reproduced and critically evaluated. Unlike a simple tutorial, the article discusses dataset construction, feature extraction, feature groups, classifier comparison, feature selection, and reproducibility.

1.2 Cybersecurity Problem

The problem addressed by the article is website phishing detection. Phishing websites imitate legitimate services such as banks, email providers, e-commerce websites, and enterprise login portals. Their purpose is to trick users into entering sensitive information such as usernames, passwords, credit-card numbers, or personal data.

The article formulates phishing detection as a supervised binary classification problem. Each website is represented by a set of extracted features, and the model predicts whether the website is legitimate or phishing. In this project, legitimate websites are encoded as class 0 and phishing websites as class 1. Treating phishing as the positive class is important because the main cybersecurity goal is detecting malicious websites.

1.3 Motivation and Proposed Solution

A major motivation of the article is that many phishing-detection studies are difficult to reproduce. Phishing websites are often short-lived, URLs may disappear quickly, webpages may change, and external information such as domain age, web traffic, DNS records, Google indexing, or page rank can change over time. This makes it hard to verify whether high reported accuracy in previous studies reflects a reliable method or a dataset-specific result.

To address this, the authors construct a reproducible and extensible benchmark dataset. They collect legitimate and phishing URLs, remove duplicated and dead URLs, limit excessive repetition from the same domain, generate DOM objects, and extract 87 features. The extracted features are organized into three groups: URL-based features, webpage-content-based features, and external-service-based features.

The article then evaluates common machine-learning classifiers including Decision Tree, Random Forest, Logistic Regression, Naive Bayes, and Support Vector Machine. The main reported conclusion is that Random Forest performs best among the tested models, especially when the different feature groups are combined.

1.4 Dataset Used in This Project

The article provides multiple dataset files for different levels of reproducibility. Dataset A contains URLs, DOM objects, and source information. It is useful for reproducing or extending the feature-extraction process. Dataset B contains extracted feature vectors in CSV format and is intended for direct machine-learning experiments.

This project uses Dataset B because the goal is to reproduce and critically evaluate the article's machine-learning experiments, not to rebuild the entire crawling and feature extraction pipeline. The notebook loaded the dataset with 11,430 rows and 89 columns. These columns include the URL identifier, 87 extracted features, and the target column.

Table 1. Dataset summary

Property	Value
Rows	11,430
Raw columns	89
Extracted features	87
Identifier columns removed	url
Final modeling features	81
Legitimate samples	5,715
Phishing samples	5,715

1.5 Main Claims Evaluated

- Random Forest is the most predictive classifier among the models tested by the authors.
- Hybrid feature sets, which combine URL, content, and external-service features, outperform individual feature groups.
- External-service features are highly discriminative for phishing detection.
- Content-based features are useful but less discriminative than URL-based or external-service-based features.
- Feature selection or feature reduction can preserve or slightly improve performance.
- Some strong features may be costly or less practical in a real-time deployment setting.

1.6 Project Scope

The project reproduces the article at the dataset-analysis and machine-learning level. It does not fully reproduce raw URL collection, live webpage crawling, DOM generation, or external-service querying. That choice is appropriate because Dataset B is the released feature-vector dataset intended for model training and evaluation.

The notebook also adds critical extensions required for a stronger cybersecurity evaluation: XGBoost comparison, richer metrics, confusion matrix analysis, class-imbalance simulation, false-positive/false-negative analysis, and deployment-cost discussion.

2. Critical Evaluation

2.1 Overall Judgment

The article is a strong and mostly reproducible study. Its core finding—that Random Forest trained on engineered phishing features performs very well—is supported by the reproduced results. The dataset is clearly structured, the feature groups are meaningful, and the model results are close to those reported by the authors.

However, the article should not be read as proving that Random Forest is the best possible phishing-detection model. It proves that Random Forest was best among the specific models tested in the paper. In this project, XGBoost slightly outperformed Random Forest, which shows that the original model comparison was valid but incomplete and back in day of publishing this article there was already better models or more advanced such as XGBoost, LightGBM.

2.2 Strengths of the Article

- The article addresses a real cybersecurity problem with clear operational importance.
- The released dataset makes the work more reproducible than many phishing-detection studies.
- The feature engineering is interpretable: URL, content, and external-service features each correspond to meaningful cybersecurity signals.
- The article evaluates multiple classifiers rather than relying on a single model.
- The article investigates feature groups and feature selection, making the analysis more informative than a simple black-box classifier comparison.

2.3 Limitations of the Article

- The model comparison is limited. The article did not evaluate stronger boosted-tree methods such as XGBoost, LightGBM, or CatBoost.
- The dataset is artificially balanced. This is useful for benchmarking, but real-world phishing prevalence is much lower than 50%.
- Some strong features depend on external services or time-sensitive reputation information, which may be slow, rate-limited, unavailable, or unstable in deployment.
- The original evaluation focuses mainly on Accuracy and Macro F1. For phishing detection, precision, recall, F2-score, MCC, ROC-AUC, and confusion matrices are also needed.
- The results are based on one constructed benchmark dataset. Additional external validation would be needed to prove generalization.

2.4 Article Experiment Alignment

Table 2. Alignment with the article’s experimental structure

Article experiment	Status in this project	Interpretation
Experiment I: classifiers and feature groups	Implemented and enhanced with pairwise feature-group combinations	Core article claim can be tested directly
Experiment II: model-combination strategies	Implemented with AND, OR, majority voting, and stacking	Tests whether separate feature-class models improve over a hybrid model
Experiment III: filter-based feature selection	Partially approximated using top-k feature importance; exact chi-square/Pearson/IG/Relief not reproduced	Supports discussion of feature reduction but is not a one-to-one reproduction
Experiment IV: wrapper-based feature selection	Not reproduced	Listed as a limitation of reproduction scope
Experiment V: feature/runtime suitability	Approximated through deployment-cost/suspicious-feature removal; exact extraction-time measurement not reproduced	Useful deployment proxy, but not the same as measuring feature extraction time

2.5 Critical Extension: XGBoost

The addition of XGBoost is an important critical extension. XGBoost is a boosted-tree ensemble method and is often strong on tabular data. In the notebook, XGBoost achieved 96.68% accuracy and 96.68% F1-score, compared with Random Forest’s 96.24% accuracy and 96.25% F1-score.

This improvement is slight, so it does not invalidate the article. Instead, it refines the conclusion. A fair statement is: Random Forest was the best model among the models tested in the article, but newer or stronger ensemble models can slightly improve the result.

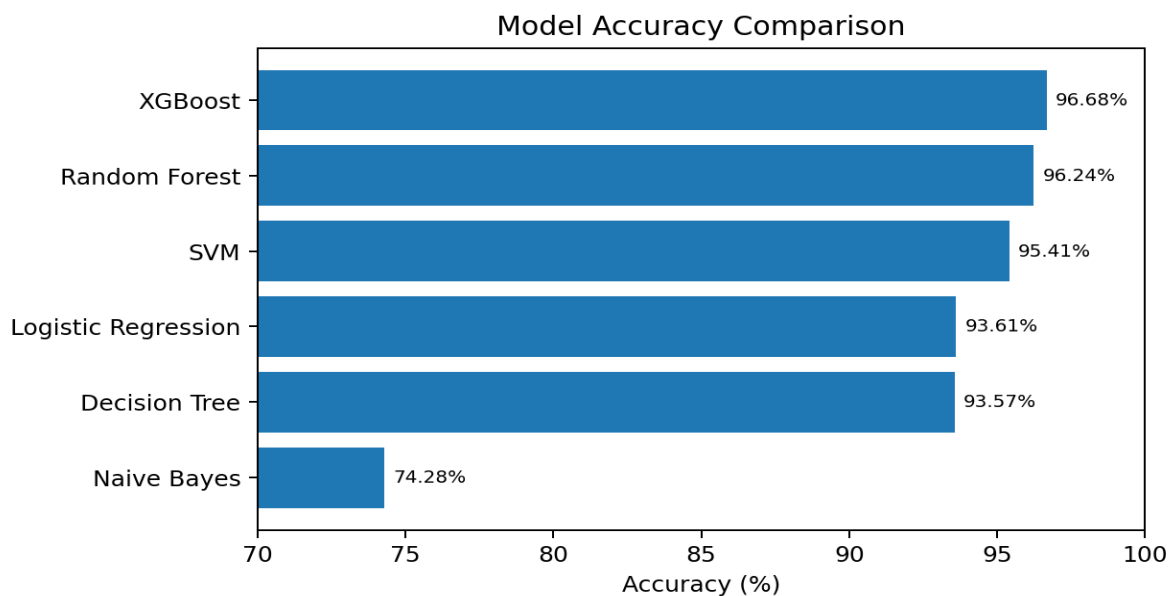


Figure 1. Model accuracy comparison. XGBoost slightly outperformed Random Forest, but Random Forest remained the strongest model among the classifiers tested in the original article.

3. Feature Engineering Analysis

3.1 Feature Engineering in the Article

Feature engineering is one of the central parts of the selected article. The authors do not train models directly on raw URLs or raw webpage text. Instead, they transform each website into a structured feature vector that can be used by classical machine-learning models such as Random Forest, Decision Tree, Logistic Regression, Naive Bayes, and SVM.

The article extracts 87 features from each website and divides them into three main groups: URL-based features, webpage-content-based features, and external-service-based features. This division is meaningful because phishing websites can show suspicious behavior at different levels. Some indicators appear in the URL structure, some appear in the webpage content, and others appear in external reputation or domain information.

3.2 URL-Based Features

URL-based features are extracted from the URL string, such as URL length, hostname length, number of dots, hyphens, digits, subdomains, HTTPS usage, IP address usage, suspicious words, shortening services, and unusual top-level domains. These features are useful because phishing URLs often use misleading structure to imitate legitimate websites. They are also fast to compute because they do not require downloading the webpage. However, attackers can sometimes create clean-looking URLs, so URL features alone may not be sufficient.

3.3 Webpage-Content-Based Features

Webpage-content features are extracted from the HTML and page structure, such as hyperlinks, internal/external link ratios, login forms, iframes, favicon behavior, media links, form actions, and page titles. These features help detect suspicious webpage behavior that may not appear in the URL. Their limitation is that they require fetching and parsing the webpage, which can be slower, unreliable, or difficult when pages are offline, dynamic, or block crawlers.

3.4 External-Service-Based Features

External-service features use information from outside sources, such as domain age, domain registration length, DNS records, web traffic, Google indexing, page rank, and WHOIS information. These features are powerful because legitimate websites usually have stronger reputation signals, while phishing websites are often new, short-lived, and less trusted. However, they have higher deployment cost because they may require external queries, introduce latency, depend on third-party services, and change over time.

3.5 Transformations, Encoding, and Model Suitability

The article's dataset is already provided as extracted feature vectors. Most features are numerical, binary, count-based, or ratio-based. This means the main transformation was performed during feature extraction: raw URLs, webpage content, DOM objects, and external-service responses were converted into structured tabular variables.

The feature engineering is mainly domain-driven rather than based on advanced transformations such as PCA or deep feature extraction. This is appropriate for the selected models. Tree-based models such as Decision Tree and Random Forest can handle binary, count, and ratio features well and do not

require feature scaling. Linear models such as Logistic Regression and SVM may be more sensitive to scaling, but the article's strongest results rely on Random Forest, which is robust to nonlinear patterns and feature interactions.

3.6 Feature Selection and Redundancy

The article also investigates feature selection. This is important because not all 87 features are equally useful. Some may be weak, redundant, expensive to compute, or unsuitable for real-time deployment.

The article evaluates filter-based methods such as chi-square, Pearson correlation, information gain, and Relief. These methods rank features according to their relationship with the target class. Feature selection is meaningful mathematically because it can reduce dimensionality, remove weak variables, improve interpretability, and sometimes reduce overfitting. From a cybersecurity perspective, it can also make the detector faster and cheaper to deploy.

Redundancy is also important because several phishing features may capture similar behavior. For example, URL length, number of special characters, number of subdomains, and path length may be correlated. Redundant features do not necessarily harm Random Forest strongly, but they can make interpretation harder and increase feature-extraction cost. In the reproduction notebook, duplicated and constant features were removed, and correlation analysis was used to identify highly related features.

3.7 Meaningfulness of the Feature Engineering

The article's feature engineering is meaningful both mathematically and from a cybersecurity perspective. Mathematically, the authors convert unstructured website information into a numerical feature vector suitable for machine learning. The features include counts, ratios, binary indicators, and reputation signals, which are appropriate for classical classifiers.

From a cybersecurity perspective, the features reflect real phishing behavior. Phishing websites often use suspicious URL structures, abnormal hyperlink patterns, misleading forms, young domains, weak reputation, and missing search-engine visibility. Therefore, the features are not arbitrary; they are connected to known phishing techniques.

The feature-group design is also valuable because it allows the article to analyze which evidence sources are most useful. The reproduced results support the article's conclusion that external-service features are highly discriminative and that combining all feature groups gives the best performance.

3.8 Limitations of the Paper's Feature Engineering

The main limitation is deployment cost. Some of the strongest features depend on external services, such as Google indexing, page rank, web traffic, WHOIS information, domain age, and DNS records. These features may be slow to collect, unavailable, rate-limited, or unstable over time.

Another limitation is that content-based features require accessing and parsing the webpage. This can be risky or unreliable if the website is offline, blocks crawlers, redirects users, or uses dynamic JavaScript. In addition, manually engineered features may miss new phishing techniques because attackers can adapt and design webpages that avoid known suspicious indicators.

The feature set may also contain correlated or redundant features. While this may not severely affect tree-based models, it can reduce interpretability and increase unnecessary complexity.

3.9 Possible Additional Features

Several additional features could improve a real phishing-detection system. These include TLS certificate age and issuer reputation, WHOIS privacy indicators, hosting provider or ASN reputation, redirect-chain analysis, JavaScript obfuscation indicators, form-action destination analysis, visual similarity to known brands, and historical behavior of the domain or IP address.

These features could improve detection, but they also introduce trade-offs. Some may increase accuracy while also increasing runtime, cost, privacy concerns, or dependency on third-party services. Therefore, future work should evaluate not only predictive performance but also feature availability, extraction cost, and deployment reliability.

3.10 Overall Assessment

Overall, the article's feature engineering is one of its strongest contributions. The authors transform raw website information into an interpretable and structured feature set, organize the features into meaningful cybersecurity categories, and evaluate which feature groups contribute most to phishing detection.

The feature engineering is effective and mostly well justified. However, the strongest features often depend on external services or time-sensitive information. This means that the best benchmark-performing feature set may not always be the easiest to deploy in a real-time system. Therefore, the article's feature engineering is strong, but future work should study feature stability, runtime cost, and robustness against adaptive phishing attacks.

4. Reproducibility Analysis

4.1 Reproducibility of the Article

A major strength of the selected article is that it provides a dataset that allows the machine-learning part of the work to be reproduced. The authors release the extracted feature dataset, referred to as Dataset B, which contains the feature vectors used for classification. This makes it possible to train models such as Random Forest and compare the reproduced results with the article's reported results.

The article is more reproducible than many phishing-detection studies because phishing URLs often disappear over time. If only the original URLs were provided, it would be difficult to recreate the same dataset later. By releasing extracted features, the authors make the core machine-learning experiment more stable and easier to reproduce.

However, the article is not fully reproducible at the complete pipeline level. Full end-to-end reproduction would require repeating the original URL collection, webpage crawling, DOM generation, and external-service feature extraction. This is difficult because phishing pages may disappear, legitimate webpages may change, and external-service values such as Google indexing, page rank, DNS records, web traffic, and domain age may change over time.

Therefore, the article is highly reproducible at the **feature-dataset and model-training level**, but less reproducible at the **raw web-crawling and feature-extraction level**.

4.2 Dataset Files and Their Role

The article provides more than one dataset file because each file supports a different level of reproducibility.

Dataset A contains URLs, DOM objects, and source information. This file is useful for studying or extending the feature-extraction process. It helps preserve webpage structure because live phishing websites may no longer be available later.

Dataset B contains the extracted numerical feature vectors in CSV format. This is the most relevant file for reproducing the machine-learning experiments because it can be loaded directly into a notebook and used for training classifiers.

This project uses Dataset B because the goal is to reproduce and critically evaluate the article's machine-learning results, not to rebuild the entire raw data collection pipeline.

4.3 Reproduction Performed in This Project

This project reproduces the article at the machine-learning experiment level. The released feature dataset was loaded, cleaned, inspected, and used to train classification models. The main reproduced model is Random Forest, since the article reports Random Forest as the strongest classifier among the models it tested.

The reproduced Random Forest result was close to the article's reported result, which supports the reproducibility of the article's main claim. The project also reproduced the article's general feature-group idea by evaluating URL-based, content-based, external-service-based, and hybrid feature sets. In addition, the notebook includes model comparison, feature importance analysis, error analysis, class-imbalance simulation, and XGBoost as an extension.

The project does not claim to reproduce every experiment from the article exactly. The article includes additional experiments such as model-combination methods, filter-based feature selection, wrapper-based feature selection, and feature extraction runtime analysis. This project focuses on the core reproducible claims most relevant to the assignment: Random Forest performance, the usefulness of feature groups, and the practical meaning of the evaluation results.

4.4 Reproducibility Limitations

Several factors prevent exact reproduction of the article.

First, the original article used Weka with default model parameters and 10-fold cross-validation, while this project uses Python, scikit-learn, and XGBoost. Differences in software libraries, random seeds, preprocessing decisions, and model defaults can lead to small differences in results.

Second, some preprocessing details are not always fully explicit. For example, decisions about duplicated features, constant features, feature scaling, or exact train/test procedures may differ between implementations.

Third, external-service features are time-sensitive. Values such as Google index, page rank, web traffic, DNS records, and domain information may not remain stable over time. This means that even if the feature-extraction code is available, rerunning it later may not produce exactly the same values.

Fourth, phishing websites are unstable by nature. Many phishing URLs are short-lived and may disappear quickly. This makes full raw-data reconstruction difficult.

4.5 Overall Reproducibility Judgment

Overall, the article is reasonably reproducible for the purpose of machine-learning evaluation. Dataset B allows the main classifier results to be reproduced without recollecting live phishing webpages. The reproduced Random Forest result was close to the article's reported result, which supports the validity of the released dataset and the main experimental claim.

At the same time, the article is only partially reproducible as a complete end-to-end cybersecurity pipeline. The raw collection process, webpage content, and external-service features depend on changing web resources. Therefore, the strongest conclusion is that the article is reproducible at the benchmark-feature-dataset level, but not perfectly reproducible at the full live-data-collection level.

5. Experimental Results

5.1 Main Model Comparison

The main model comparison trained Logistic Regression, Decision Tree, Random Forest, SVM, Naive Bayes, and XGBoost. Among the models tested in the original article, Random Forest performed best. XGBoost, added as a project extension, achieved the best overall score.

Table 10. Main model comparison

model	accuracy	precision	recall	f1	f2	roc_auc	mcc
XGBoost	96.68%	96.43%	96.94%	96.68%	96.84%	99.42%	93.35%
Random Forest	96.24%	95.92%	96.59%	96.25%	96.45%	99.34%	92.48%
SVM	95.41%	95.77%	95.01%	95.39%	95.16%	98.83%	90.82%
Logistic Regression	93.61%	93.92%	93.26%	93.59%	93.39%	98.14%	87.23%
Decision Tree	93.57%	93.53%	93.61%	93.57%	93.60%	93.57%	87.14%
Naive Bayes	74.28%	69.20%	87.49%	77.28%	83.10%	80.51%	50.35%

5.2 Random Forest Reproduction

Random Forest is the main reproduction model because it is the model emphasized in the article. In the notebook, Random Forest achieved 96.24% accuracy, 95.92% precision, 96.59% recall, 96.25% F1-score, 99.34% ROC-AUC, and 92.48% MCC. This is close to the article's reported Random Forest performance of approximately 96.61% accuracy using hybrid features.

The Random Forest confusion matrix shows 47 false positives and 39 false negatives. In phishing detection, false negatives are especially dangerous because they allow phishing websites to be treated as legitimate. False positives are also important because they can block legitimate websites and create user distrust.

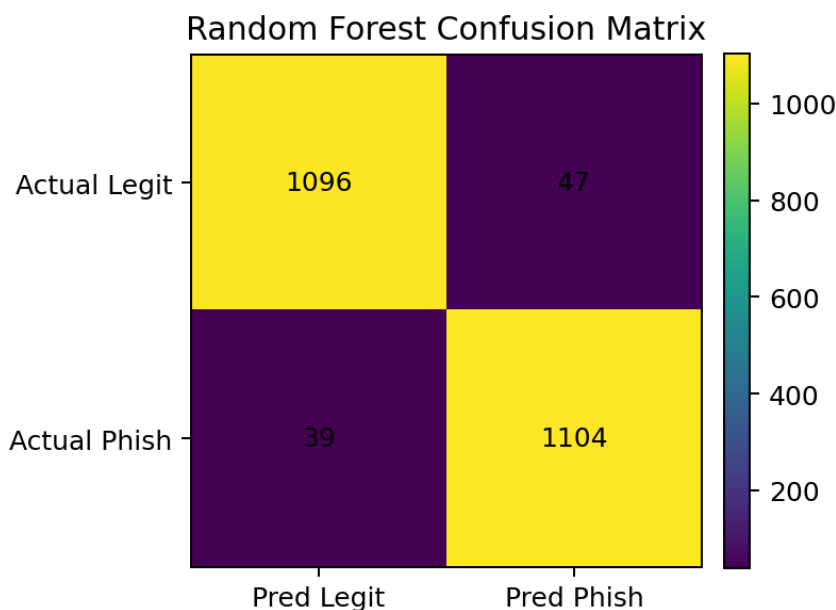


Figure 3. Random Forest confusion matrix on the balanced test set.

5.3 Experiment I: Feature Groups and Pairwise Combinations

The article’s Experiment I compares feature groups and feature-group combinations. The notebook implements this more directly by testing URL-only, Content-only, External-only, URL + Content, URL + External, Content + External, and All Features. The strongest Random Forest feature configuration was URL + External with 96.41% accuracy, slightly above All Features at 96.24%. This aligns with the article’s observation that external-service features and hybrid combinations are highly effective.

Table 11. Experiment I: Random Forest feature-group and pairwise-combination results

feature_group	num_features	accuracy	precision	recall	f1	roc_auc	mcc
URL only	55	90.99%	90.49%	91.60%	91.04%	97.04%	81.98%
Content only	19	90.42%	90.67%	90.11%	90.39%	96.07%	80.84%
External only	7	93.66%	93.39%	93.96%	93.68%	97.63%	87.32%
URL + Content	74	94.44%	95.12%	93.70%	94.40%	98.39%	88.90%
URL + External	62	96.41%	95.85%	97.03%	96.43%	99.38%	92.83%
Content + External	26	94.75%	94.83%	94.66%	94.75%	98.68%	89.50%
All Features	81	96.24%	95.92%	96.59%	96.25%	99.34%	92.48%

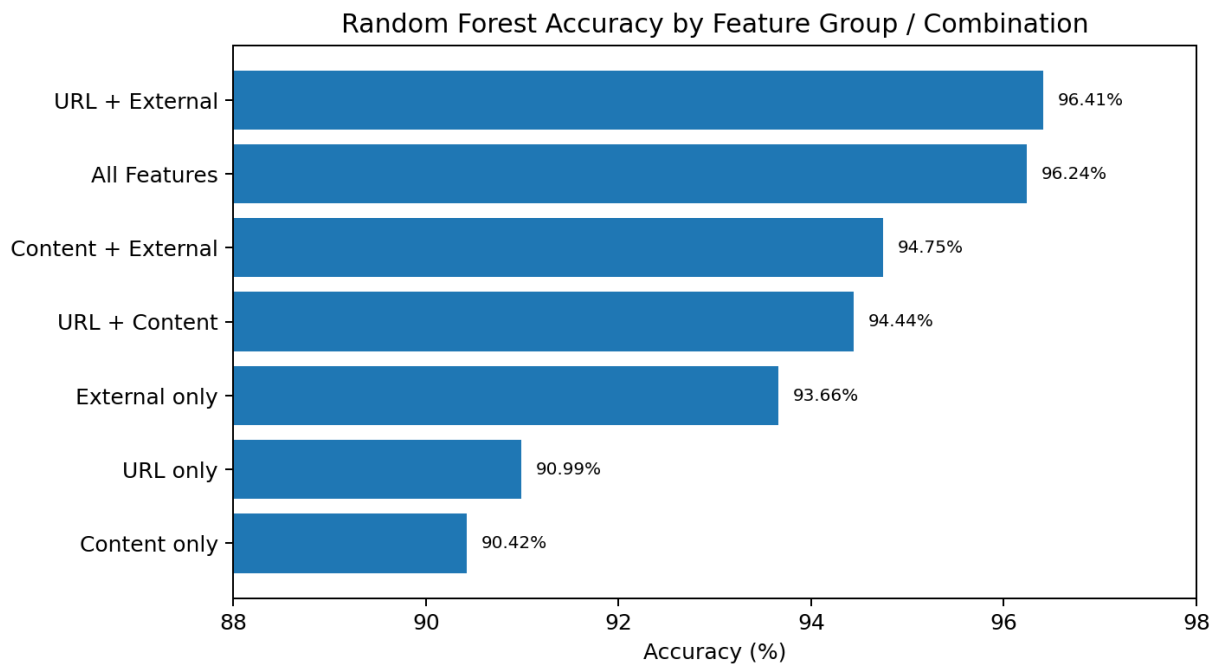


Figure 4. Random Forest accuracy across feature groups and pairwise combinations.

The first part of Experiment I compares the performance of several classifiers on the full cleaned feature set. This follows the article’s goal of identifying which classifier is most suitable for phishing website detection. In this reproduction, Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost were trained and evaluated using the same train/test split. The comparison shows that ensemble tree-based models perform better than simpler models. Random Forest achieved results close to the article’s reported performance, which supports the paper’s conclusion that Random Forest is highly effective for this dataset. XGBoost achieved slightly higher performance in the reproduced experiment, suggesting that the article’s conclusion is valid for the models it tested, but may not be exhaustive when stronger modern boosting models are included.

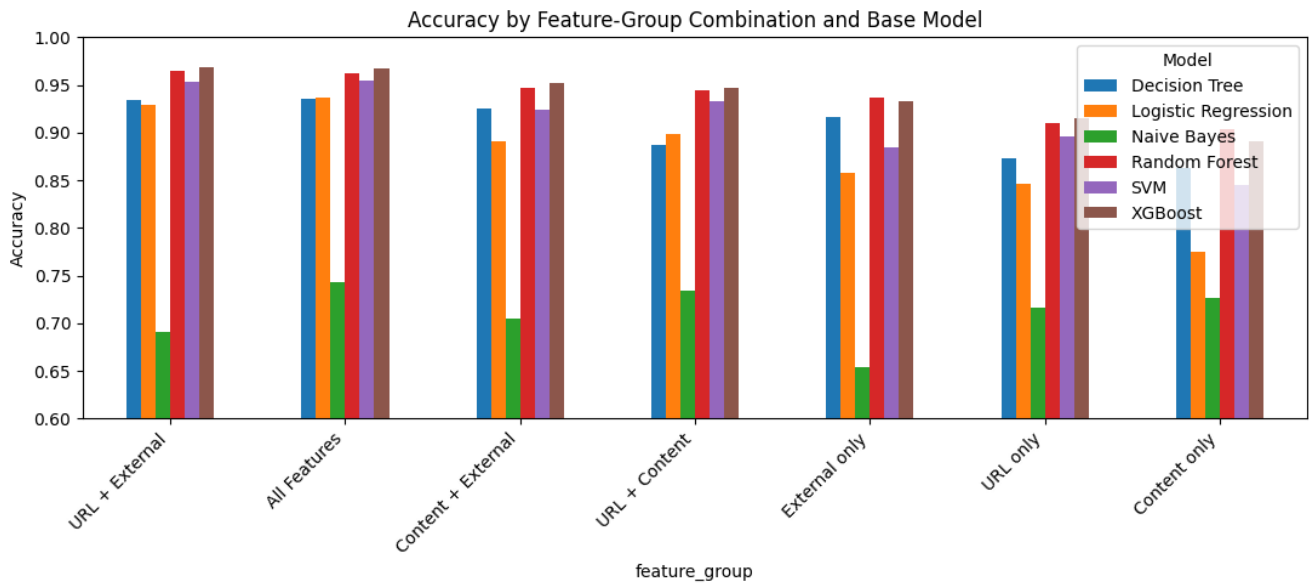


Figure 5. classifier comparison on the full cleaned feature set

5.4 Experiment II: Combining Feature-Class Models

Experiment II was updated to match the final notebook implementation. Instead of testing only Random Forest, the reproduction trains separate URL-feature, content-feature, and external-service-feature models for each base classifier, then combines their predictions using four strategies: AND, OR, majority voting, and stacking. AND is the strictest rule because it predicts phishing only when all three specialized models agree. OR is the most aggressive rule because it predicts phishing when any specialized model flags the website. Majority voting requires agreement from at least two models, while stacking uses the outputs of the specialized models as inputs to a meta-classifier.

The results show a clear pattern. Stacking gives the best accuracy for every tested base model, with XGBoost stacking reaching 96.68% accuracy and Random Forest stacking reaching 96.63%. Majority voting is consistently the second-best balanced strategy. AND produces very high precision but lower recall, meaning it reduces false alarms but misses more phishing websites. OR produces very high recall but lower precision, meaning it catches more phishing attempts at the cost of more false positives. Therefore, Experiment II is useful not only as a model-combination experiment, but also as a cybersecurity trade-off analysis between precision-oriented and recall-oriented detection.

Table 8. Experiment II model-combination results across base classifiers

Base model	Method	Acc.	Prec.	Recall	F1	F2	ROC-AUC	MCC
Logistic Regression	AND	80.31%	97.79%	62.03%	75.91%	66.92%	97.53%	65.14%
Logistic Regression	OR	77.25%	69.19%	98.25%	81.20%	90.64%	97.53%	60.06%
Logistic Regression	Majority voting	90.42%	88.95%	92.30%	90.60%	91.61%	97.53%	80.90%
Logistic Regression	Stacking	92.48%	92.03%	93.00%	92.52%	92.81%	97.60%	84.96%
Decision Tree	AND	87.23%	98.74%	75.42%	85.52%	79.16%	97.57%	76.62%
Decision Tree	OR	84.03%	76.11%	99.21%	86.14%	93.53%	97.57%	71.44%
Decision Tree	Majority voting	93.96%	93.43%	94.58%	94.00%	94.34%	97.57%	87.93%
Decision Tree	Stacking	94.01%	93.51%	94.58%	94.04%	94.36%	97.77%	88.02%
Random Forest	AND	90.03%	99.25%	80.66%	89.00%	83.80%	99.34%	81.49%
Random Forest	OR	89.68%	83.27%	99.30%	90.58%	95.62%	99.34%	80.86%
Random Forest	Majority voting	95.36%	95.05%	95.71%	95.38%	95.58%	99.34%	90.73%
Random Forest	Stacking	96.63%	96.35%	96.94%	96.64%	96.82%	99.36%	93.27%
SVM	AND	85.87%	98.69%	72.70%	83.73%	76.75%	98.57%	74.37%
SVM	OR	82.90%	74.83%	99.13%	85.28%	93.08%	98.57%	69.56%
SVM	Majority voting	93.70%	92.29%	95.36%	93.80%	94.73%	98.57%	87.45%
SVM	Stacking	94.27%	93.32%	95.36%	94.33%	94.95%	98.59%	88.56%
Naive Bayes	AND	65.05%	98.04%	30.71%	46.77%	35.60%	89.24%	41.41%
Naive Bayes	OR	67.15%	60.46%	99.13%	75.11%	87.88%	89.24%	44.61%
Naive Bayes	Majority voting	77.43%	76.96%	78.30%	77.62%	78.03%	89.24%	54.86%
Naive Bayes	Stacking	80.45%	79.69%	81.71%	80.69%	81.30%	89.40%	60.91%
XGBoost	AND	89.55%	99.24%	79.70%	88.40%	82.97%	99.28%	80.67%
XGBoost	OR	88.85%	82.17%	99.21%	89.89%	95.26%	99.28%	79.42%
XGBoost	Majority voting	95.67%	95.63%	95.71%	95.67%	95.70%	99.28%	91.34%
XGBoost	Stacking	96.68%	96.68%	96.68%	96.68%	96.68%	99.30%	93.35%

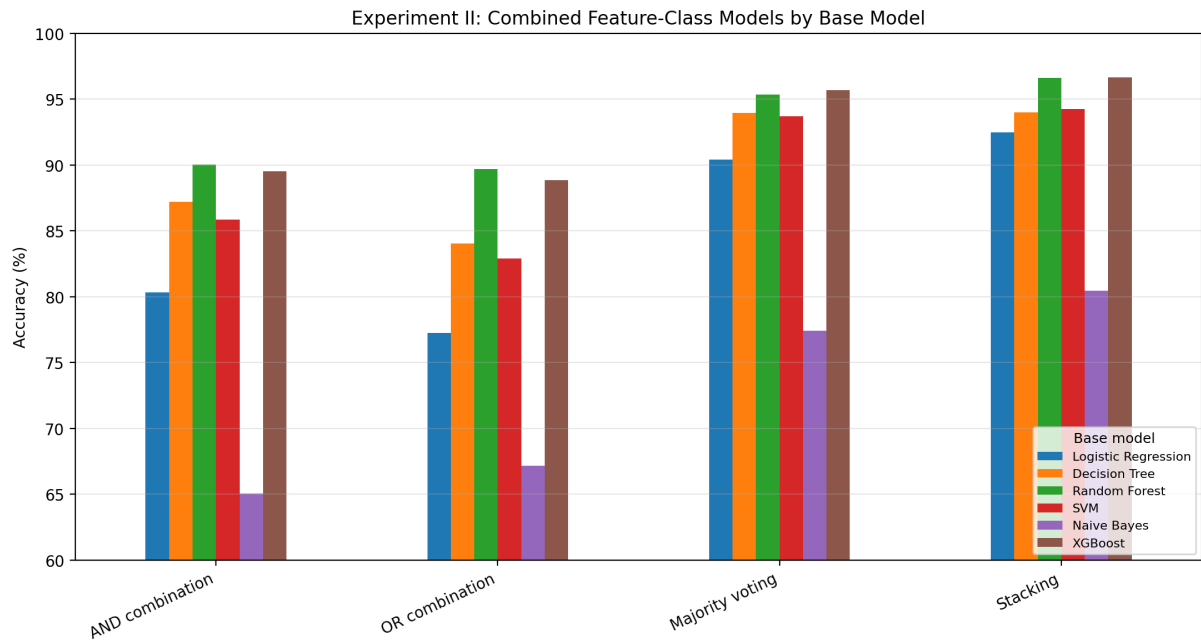


Figure 5. Experiment II combined feature-class models by base classifier.

Overall, the final Experiment II results mostly support the article's conclusion that model-combination strategies are not automatically superior to a strong model trained on the full hybrid feature set. Stacking is competitive and slightly improves the reproduced Random Forest score, but the improvement is small and depends on implementation details. The safest interpretation is that specialized feature-class combinations are useful for understanding decision trade-offs, while the all-feature ensemble remains a strong and simpler baseline

5.5 Realistic Class-Imbalance Simulation

The original dataset is balanced, but real web traffic is not. To test a more realistic scenario, in notebook we created an imbalanced test set with approximately 95% legitimate websites and 5% phishing websites. Random Forest maintained high recall of 96.67%, but precision dropped to 55.24%. This means the model still detected most phishing websites, but many phishing alerts became false positives.

Table 13. Random Forest on an imbalanced test set with approximately 5% phishing prevalence

accuracy	precision	recall	f1	f2	mcc	roc_auc
95.93%	55.24%	96.67%	70.30%	84.06%	71.38%	99.25%

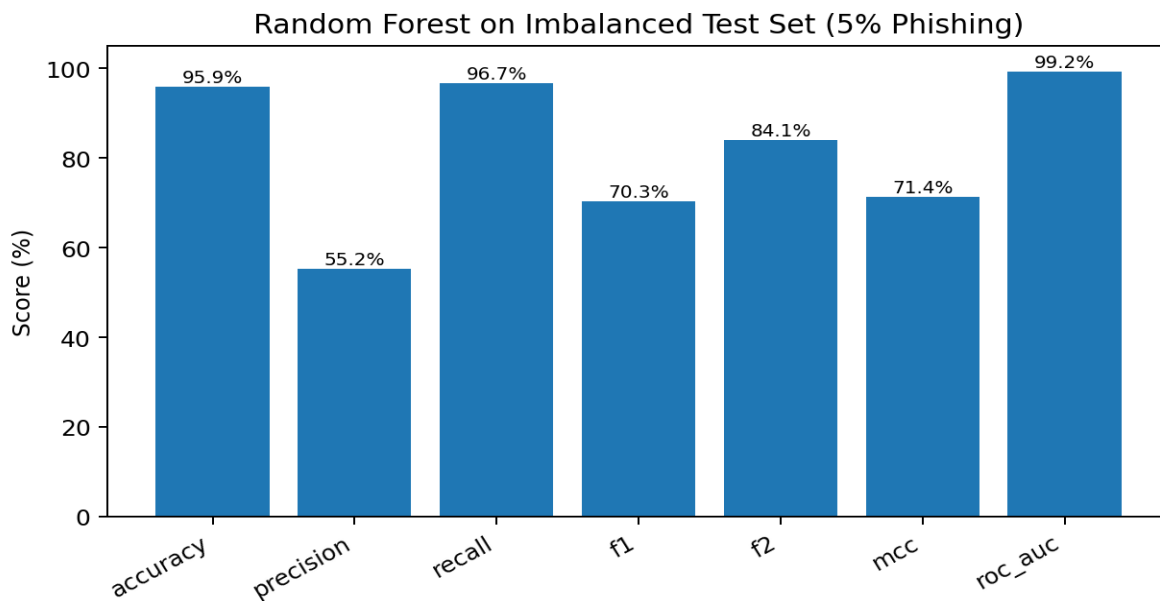


Figure 6. Under realistic class imbalance, recall remains high but precision falls sharply.

5.6 Error Analysis

The notebook compares false negatives to correctly detected phishing websites. The largest differences appeared in web_traffic, domain_age, nb_hyperlinks, domain_registration_length, length_url, links_in_tags, ratio_intmedia, ratio_extmedia, longest_word_path, and safe_anchor. This suggests that missed phishing websites may look more legitimate: they may have less suspicious URL structure, different hyperlink patterns, or reputation signals that resemble legitimate websites.

Table 14. Largest mean differences between true positives and false negatives

feature	true_positive_mean	false_negative_mean	absolute_difference
web_traffic	934104.00	664336.46	269767.54
domain_age	2990.00	2466.26	523.74
nb_hyperlinks	29.48	116.08	86.60
domain_registration_length	371.31	446.28	74.97
length_url	77.13	36.18	40.95
links_in_tags	44.70	66.88	22.17
ratio_intmedia	34.08	53.14	19.06
ratio_extmedia	17.46	34.04	16.58
longest_word_path	16.88	3.90	12.98
safe_anchor	31.83	41.79	9.95

6. Conclusions

6.1 Key Findings

- The article's central claim is mostly supported: Random Forest is a strong classifier for phishing detection and reproduces the reported performance closely.
- Among the original article's model set, Random Forest is the strongest model in the notebook.
- XGBoost slightly outperforms Random Forest, showing that the article's model comparison was not exhaustive.
- External-service features are highly discriminative. Seven external-service features alone achieve 93.66% Random Forest accuracy.
- Hybrid combinations are powerful. URL + External features achieve 96.41% Random Forest accuracy.
- Stacking specialized feature-class models achieves performance comparable to or slightly above the all-feature Random Forest model.
- Benchmark accuracy is not enough for cybersecurity deployment. Under 5% phishing prevalence, precision drops to 55.24% even though recall stays high.

6.2 Answer to the Research Questions

Table 15. Research-question summary

Research question	Answer
Can the Random Forest result be reproduced?	Yes. Random Forest achieved 96.24% accuracy, close to the article's reported result.
Are feature groups important?	Yes. Hybrid and pairwise combinations outperform most individual groups.
Are external-service features discriminative?	Yes. External-only Random Forest achieved 93.66% accuracy with only seven features.
Does model combination help?	Partially. Random Forest stacking achieved 96.63%, slightly above the all-feature RF result.
Does XGBoost challenge the paper?	Yes, slightly. XGBoost achieved 96.68% accuracy.
Are balanced-dataset metrics sufficient for cybersecurity?	No. The imbalance simulation showed precision drops sharply under realistic prevalence.

6.3 Practical Cybersecurity Interpretation

The most important cybersecurity lesson is that phishing detection should not be judged by accuracy alone. A detector can show high accuracy on a balanced benchmark and still produce too many false positives when deployed in real traffic. This project's imbalance simulation demonstrates that recall can remain high while precision collapses.

For a real phishing-detection system, the decision threshold should be tuned according to the deployment goal. If the system is used as a warning layer, high recall may be preferred. If the system blocks websites automatically, precision becomes more important to avoid blocking legitimate websites.

6.4 Suggested Improvements

- Compare Random Forest with XGBoost, LightGBM, and CatBoost in the original paper's evaluation.
- Evaluate realistic class imbalance and threshold tuning.
- Report MCC, F2-score, precision-recall curves, and confusion matrices in addition to accuracy and F1.
- Measure actual feature extraction time for URL, content, and external-service features.
- Evaluate fallback models that avoid external services.
- Use temporal validation to test whether models trained on older phishing data detect newer phishing websites.
- Test the model on at least one external dataset to evaluate generalization.

6.5 Final Conclusion

This project shows that the selected article is strong, reproducible, and mostly supported by the data. Random Forest performs very well on the released dataset, and the feature-group results confirm the importance of hybrid features and external-service signals.

At the same time, the project demonstrates that the article's conclusions should be interpreted carefully. Random Forest was the best among the models tested in the article, but not necessarily the best possible model. XGBoost slightly improves the result, and stacking feature-class models also performs strongly. Realistic class imbalance changes the interpretation of the metrics, making precision and false-positive management central deployment concerns.

Overall, the article provides a valuable benchmark and a meaningful foundation for machine-learning-based phishing detection. The best final judgment is that the paper's main claims are reproducible and well supported on its own dataset, but a production-ready phishing detector would require stronger model comparison, realistic prevalence testing, runtime analysis, and external validation.

References

- [1] A. Hannousse and S. Yahiouche, "Towards Benchmark Datasets for Machine Learning Based Website Phishing Detection: An Experimental Study," 2020.
- [2] A. Hannousse and S. Yahiouche, "Web page phishing detection dataset," Mendeley Data, 2020.
- [3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
- [4] Project notebook: `phishing_detection_project.ipynb`, final executed version supplied for this report.